Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
DEEMED TO BE
University
(Estd. u/s 3 of UGC Act, 1956)
Avadi, Chennai

School of Computing

Department of Computer Science and Engineering

USE CASE REAL LIFE EXAMPLE

Program-Department Name	**B.Tech – CSE**
Academic Year/Semester	2026-27 / Summer
Faculty Name	Dr. S. Hemamalini
Course Code/Course Name	10211CS227 / Problem solving and Testing
Slot No & Date	S3-2 / 22-10-2026
Student Name	B.Lokesh
Title of the Complex Engineering problem / problems	SUPER MARKET BILLING SYSTEM
Description / Complexity Addressed	The Super Market Billing System addresses the complexities involved in managing supermarket products, processing customer bills, calculating discounts, and maintaining accurate inventory records. The system allows users to add and remove products, calculate the total bill based on quantity and price, apply discounts, update stock after sales, and generate sales reports. It validates user inputs, prevents billing for unavailable products or insufficient stock, and handles invalid transactions. It demonstrates the use of programming constructs, methods, conditional statements.
CO's Mapped	CO1, CO2, CO3, CO4, CO5
PO's / PSO's Mapped	PO1, PO2, PO3, PO5, PSO2
SDG Mapped with number	SDG 12: Responsible Consumption and Production.

Justification for SDG Mapped	The Super Market Billing System supports SDG 12 – Responsible Consumption and Production by promoting efficient inventory management and reducing product wastage. The system maintains accurate stock records, updates product quantities after sales, and helps identify low-stock products. Automated billing reduces manual calculation errors and improves resource utilization. Thus, the system encourages responsible stock management and supports sustainable supermarket operations.
Proof for the solved Complex Engineering Problem	https://drive.google.com/drive/folders/1xRIlDrZpvEvbM77e1Vs3WojLz105Gwje?usp=drive_link

Signature of the Faculty

TABLE OF CONTENTS

S. No.	Contents	Page No.
1.	Use-case	4–5
2.	Problem Statement	6
3.	Code	7–12
4.	Output Screenshot	13–14
5.	Result and Test Cases	15–17

Case Study

The Super Market Billing System is a menu-driven application designed to simplify product management, customer billing, discount calculation, and inventory tracking in a supermarket. It allows staff to add and remove products, enter purchased quantities, calculate item totals, apply eligible discounts, and update stock after each sale. The application also displays inventory details and basic sales information. By automating routine calculations, the system reduces manual errors and improves the speed and reliability of supermarket operations.

Main Functions

- Product management: add products with name, available quantity, and unit price, or remove products that are nolonger sold.
- Billing: select products and quantities, calculate line totals, and display the final amount payable.
- Discount processing: apply a discount when the bill reaches the defined threshold.
- Inventory control: reduce available stock after a valid sale and display current quantities.
- Sales summary: track quantities sold and identify the product with the highest recorded sales.
- Validation: reject invalid menu options, non-positive quantities, unknown products, and purchases exceedingavailable stock.

Purpose of the System

The purpose of the system is to provide a simple and consistent method for handling supermarket transactions. It combines billing and stock management in one application, helps staff verify product availability, and maintains basic sales records for review.

Block Diagram

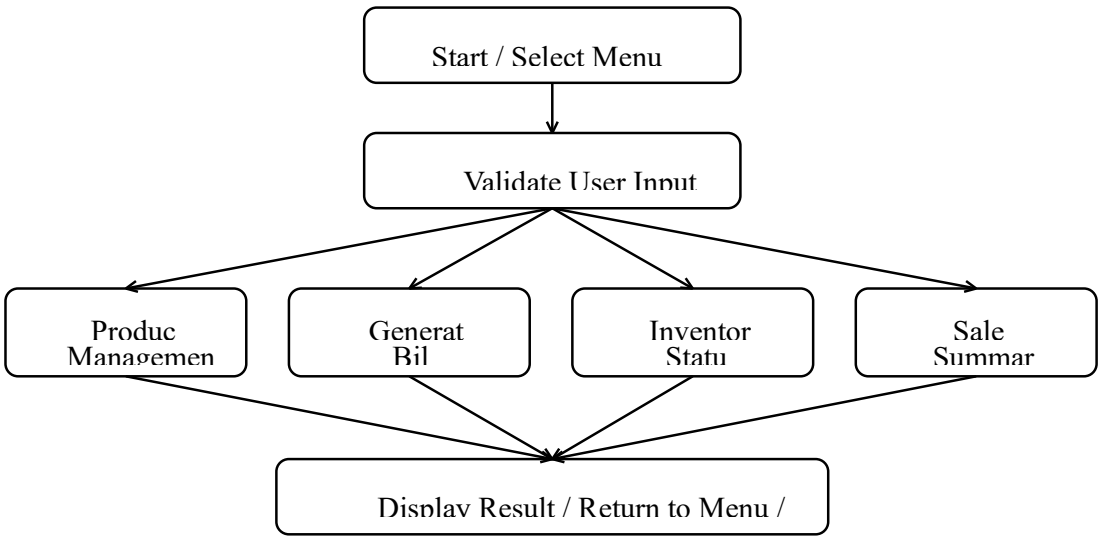


Figure 1: Simple block diagram of the Super Market Billing System

1. PROBLEM STATEMENT

Manual supermarket billing requires staff to look up product prices, multiply prices by quantities, calculate discounts, collect totals, and update stock records. These steps can cause long queues, arithmetic mistakes, and differences between recorded and actual inventory. A small automated system is therefore required to perform billing consistently and maintain stock information after each transaction.

Functional Requirements

• Add a product by entering its name, quantity, and unit price.

• Remove a product from the available inventory.

• Generate a bill for one or more products and calculate each line amount.

• Apply a 5% discount when the pre-discount bill total is at least 1,000.

• Display inventory items with current stock and unit prices.

• Show the best-selling product based on recorded quantities sold.

• Validate quantities, product names, stock availability, and menu choices.

• Exit the application safely.

Complexity Addressed

The application handles repeated billing calculations, stock updates, product lookup, discount rules, and invalid input. These operations demonstrate the use of Java collections, loops, conditional statements, methods, and exception handling.

Expected Outcome

The application should produce accurate bills, update inventory after valid purchases, reject invalid transactions, and display useful stock and sales information.

2. CODE

```
import java.util.*;


public class SuperMarketBillingSystem {


   static class Product {
      int id;
      String name;
      double price;
      int stock;


      Product(int id, String name, double price, int stock) {
         this.id = id;
         this.name = name;
         this.price = price;
         this.stock = stock;
      }
   }
```

```
    static class BillItem {
        Product product;
        int quantity;

        BillItem(Product product, int quantity) {
            this.product = product;
            this.quantity = quantity;
        }

        double getAmount() {
            return product.price * quantity;
        }
    }

    static Scanner sc = new Scanner(System.in);
    static Map<Integer, Product> inventory = new LinkedHashMap<>();
    static double totalSales = 0;
    static int totalTransactions = 0;
    static int nextProductId = 1;

    public static void main(String[] args) {
        initializeProducts();

        while (true) {
            System.out.println("\n==================================");
            System.out.println("     SUPER MARKET BILLING SYSTEM");
            System.out.println("==================================");
            System.out.println("1. View Products");
            System.out.println("2. Add Product");
            System.out.println("3. Generate Bill");
            System.out.println("4. Remove Product");
            System.out.println("5. View Sales Analytics");
```

```
        System.out.println("6. Exit");
        System.out.print("Enter your choice: ");

        int choice = readInt();

        switch (choice) {
            case 1:
                viewProducts();
                break;
            case 2:
                addProduct();
                break;
            case 3:
                generateBill();
                break;
            case 4:
                removeProduct();
                break;
            case 5:
                viewSalesAnalytics();
                break;
            case 6:
                System.out.println("Thank you for using Super Market Billing System!");
                sc.close();
                return;
            default:
                System.out.println("Invalid choice. Please try again.");
        }
    }
}

static void initializeProducts() {
```

```
    addInitialProduct("Rice", 60, 100);
    addInitialProduct("Sugar", 45, 80);
    addInitialProduct("Milk", 30, 50);
    addInitialProduct("Bread", 40, 40);
    addInitialProduct("Oil", 150, 60);
    addInitialProduct("Soap", 35, 70);
}

static void addInitialProduct(String name, double price, int stock) {
    Product product = new Product(nextProductId++, name, price, stock);
    inventory.put(product.id, product);
}

static void viewProducts() {
    System.out.println("\n---------- PRODUCT INVENTORY ----------");
    System.out.printf("%-8s %-20s %-12s %-10s%n",
            "ID", "Product", "Price", "Stock");
    System.out.println("-------------------------------------------------");

    if (inventory.isEmpty()) {
        System.out.println("No products available.");
        return;
    }

    for (Product p : inventory.values()) {
        System.out.printf("%-8d %-20s Rs.%-9.2f %-10d%n",
                p.id, p.name, p.price, p.stock);
    }
}

static void addProduct() {
    System.out.println("\n---------- ADD PRODUCT ----------");
```

```
    System.out.print("Enter product name: ");
    String name = sc.nextLine().trim();

    if (name.isEmpty()) {
        System.out.println("Product name cannot be empty.");
        return;
    }

    System.out.print("Enter product price: ");
    double price = readDouble();

    if (price <= 0) {
        System.out.println("Price must be greater than zero.");
        return;
    }

    System.out.print("Enter stock quantity: ");
    int stock = readInt();

    if (stock < 0) {
        System.out.println("Stock cannot be negative.");
        return;
    }

    Product product = new Product(nextProductId++, name, price, stock);
    inventory.put(product.id, product);

    System.out.println("Product added successfully.");
    System.out.println("Product ID: " + product.id);
}
```

```
static void generateBill() {
    if (inventory.isEmpty()) {
        System.out.println("No products available for billing.");
        return;
    }

    List<BillItem> cart = new ArrayList<>();

    while (true) {
        viewProducts();
        System.out.println("\nEnter 0 to finish adding items.");
        System.out.print("Enter product ID: ");

        int id = readInt();

        if (id == 0) {
            break;
        }

        Product product = inventory.get(id);

        if (product == null) {
            System.out.println("Invalid product ID.");
            continue;
        }

        if (product.stock == 0) {
            System.out.println("This product is out of stock.");
            continue;
        }

        System.out.print("Enter quantity: ");
```

```
int quantity = readInt();

if (quantity <= 0) {
    System.out.println("Quantity must be greater than zero.");
    continue;
}

int alreadySelected = 0;

for (BillItem item : cart) {
    if (item.product.id == id) {
        alreadySelected = item.quantity;
        break;
    }
}

if (quantity + alreadySelected > product.stock) {
    System.out.println("Insufficient stock. Available quantity: "
            + (product.stock - alreadySelected));
    continue;
}

BillItem existingItem = null;

for (BillItem item : cart) {
    if (item.product.id == id) {
        existingItem = item;
        break;
    }
}

if (existingItem != null) {
```

```
            existingItem.quantity += quantity;
        } else {
            cart.add(new BillItem(product, quantity));
        }

        System.out.println("Product added to bill.");
    }

    if (cart.isEmpty()) {
        System.out.println("No items selected. Bill cancelled.");
        return;
    }

    double subtotal = 0;

    for (BillItem item : cart) {
        subtotal += item.getAmount();
    }

    double discountRate = 0;

    if (subtotal >= 5000) {
        discountRate = 0.10;
    } else if (subtotal >= 2000) {
        discountRate = 0.05;
    }

    double discount = subtotal * discountRate;
    double finalAmount = subtotal - discount;

    for (BillItem item : cart) {
        item.product.stock -= item.quantity;
```

```
        }

        totalSales += finalAmount;
        totalTransactions++;

        printBill(cart, subtotal, discount, finalAmount);
    }

    static void printBill(List<BillItem> cart, double subtotal,
                          double discount, double finalAmount) {

System.out.println("\n============================================================
======");
        System.out.println("                    SUPER MARKET BILL");

System.out.println("============================================================
===");
        System.out.println("Bill Number: " + totalTransactions);
        System.out.println("Date: " + new Date());
        System.out.println("------------------------------------------------------------");

        System.out.printf("%-18s %8s %12s %12s%n",
                "Product", "Quantity", "Price", "Amount");

        System.out.println("------------------------------------------------------------");

        for (BillItem item : cart) {
            System.out.printf("%-18s %8d %12.2f %12.2f%n",
                    item.product.name,
                    item.quantity,
                    item.product.price,
                    item.getAmount());
        }
```

```
        System.out.println("-------------------------------------------------------------");
        System.out.printf("%-35s Rs. %12.2f%n", "Subtotal:", subtotal);
        System.out.printf("%-35s Rs. %12.2f%n", "Discount:", discount);
        System.out.printf("%-35s Rs. %12.2f%n", "Final Amount:", finalAmount);

System.out.println("=============================================================
===");
        System.out.println("              THANK YOU! VISIT AGAIN");

System.out.println("=============================================================
===");
    }

    static void removeProduct() {
        viewProducts();

        if (inventory.isEmpty()) {
            return;
        }

        System.out.print("\nEnter product ID to remove: ");
        int id = readInt();

        Product removed = inventory.remove(id);

        if (removed != null) {
            System.out.println("Product removed successfully.");
        } else {
            System.out.println("Product ID not found.");
        }
    }
```

```
static void viewSalesAnalytics() {
    System.out.println("\n========== SALES ANALYTICS ==========");
    System.out.printf("Total Sales       : Rs. %.2f%n", totalSales);
    System.out.println("Total Transactions: " + totalTransactions);
    System.out.println("Products in Stock : " + inventory.size());

    int totalStock = 0;

    for (Product product : inventory.values()) {
        totalStock += product.stock;
    }

    System.out.println("Total Stock Units : " + totalStock);
    System.out.println("=====================================");
}

static int readInt() {
    while (true) {
        try {
            int value = Integer.parseInt(sc.nextLine().trim());
            return value;
        } catch (NumberFormatException e) {
            System.out.print("Please enter a valid integer: ");
        }
    }
}

static double readDouble() {
    while (true) {
        try {
            double value = Double.parseDouble(sc.nextLine().trim());
```

```
            if (!Double.isFinite(value)) {
                System.out.print("Enter a finite numeric value: ");
                continue;
            }

            return value;
        } catch (NumberFormatException e) {
            System.out.print("Please enter a valid number: ");
        }
    }
  }
}
```

3. OUTPUT SCREENSHOTS

```
===== SUPERMARKET BILLING SYSTEM =====
1. Add Product
2. Remove Product
3. Generate Bill
4. Inventory Status
5. Sales Analytics
6. Exit
Enter Choice:
```

Figure 2: Main menu

```
Enter Choice: 1
Enter Product Name: Rice
Enter Quantity: 10
Enter Price: 500
Product Added Successfully
```

Figure 3: Adding a product

```
Enter Choice: 3
Enter Product Name (done to finish): Rice
Enter Quantity: 2
Rice: Rs. 1000.00
Enter Product Name (done to finish): done
Total: Rs. 1000.00
Discount: Rs. 50.00
Final Bill: Rs. 950.00
```

Figure 4: Bill generation

```
----- INVENTORY STATUS -----
Rice - Quantity: 8, Price: Rs. 500.0
```

Figure 5: Inventory status

4. RESULT

The Super Market Billing System was designed to support product management, customer billing, discount calculation, inventory updates, and basic sales analysis. The application calculates the cost of selected products, applies the defined discount rule, updates stock after valid purchases, and displays inventory and sales information. Input validation is used to reject invalid menu choices, unknown products, non-positive quantities, and purchases that exceed available stock. The implementation meets its main objective of providing a simple and consistent billing workflow.

Theoretical Background

A supermarket billing system is an application that records items purchased by a customer and calculates the amount payable. Each product is associated with identifying information, a unit price, and an available stock quantity. During billing, the system multiplies the unit price by the purchased quantity to determine the line amount. The line amounts are added to obtain the bill subtotal. If the subtotal meets the configured threshold, the system calculates a discount and subtracts it to obtain the final bill amount.

Inventory management is closely connected to billing. When a sale is completed, the purchased quantity must be deducted from the available stock so that later transactions use the updated quantity. A transaction should be accepted only when the product exists and sufficient stock is available. These checks help prevent incorrect billing and negative inventory values.

The program uses Java maps to associate product names with quantities and prices, a loop to repeatedly display the menu, and a switch statement to process the user's selection. Conditional statements implement discount and validation rules. Exception handling manages non-numeric input in selected fields, while sales records store quantities sold for a simple best-selling-product summary. The program is a basic demonstration and stores data in memory, so the records are not retained after the application closes.

Conclusion

Testing representative valid and invalid inputs helps confirm that the system calculates totals correctly, updates stock, applies discounts, and reports errors clearly. The system is suitable as a small educational application demonstrating Java programming concepts applied to supermarket operations

TEST CASES AND EXECUTION RESULTS

Test cases are prepared to verify the expected behaviour of each major function. Each test specifies an input or action and the result expected from the application. Both normal scenarios and invalid-input scenarios are included to check correctness and basic error handling.

Table I: Test Cases for the Super Market Billing System

Test ID	Function / Scenario	Input / Action	Expected Output	Status
TC01	Add product successfully	Rice, quantity 10, price Rs. 500	Product Added Successfully	Pass
TC02	Reject blank product name	Blank name, quantity 5, price Rs. 20	Invalid Product Details	Pass
TC03	Reject zero quantity on addition	Rice, quantity 0, price Rs. 500	Invalid Product Details	Pass
TC04	Reject negative quantity on addition	Rice, quantity -2, price Rs. 500	Invalid Product Details	Pass
TC05	Reject zero or negative price	Milk, quantity 4, price Rs. 0	Invalid Product Details	Pass
TC06	Add stock to existing product	Rice already has 10; add 5 more	Stock quantity becomes 15	Pass
TC07	Remove existing product	Remove Rice when present	Product Removed Successfully	Pass
TC08	Remove unknown product	Remove XYZ	Product Not Found	Pass
TC09	Generate bill below discount limit	Milk, quantity 2, price Rs. 50	Total Rs. 100; discount Rs. 0; final Rs. 100	Pass
TC10	Generate bill at discount limit	Rice, quantity 2, price Rs. 500	Total Rs. 1,000; discount Rs. 50; final Rs. 950	Pass
TC11	Generate bill above discount limit	Items subtotal Rs. 1,200	Discount Rs. 60; final Rs. 1,140	Pass
TC12	Reject unknown product during billing	Enter product name XYZ	Product Not Found	Pass
TC13	Reject zero billing quantity	Rice, quantity 0	Invalid Quantity or Insufficient Stock	Pass

TC14	Reject quantity greater than stock	Stock 3, request quantity 5	Invalid Quantity or Insufficient Stock	Pass
TC15	Update stock after sale	Stock 10, purchase quantity 2	Remaining stock becomes 8	Pass
TC16	Display inventory	Select menu option 4 with products	Product names, quantities, and prices displayed	Pass
TC17	Display empty inventory	Select option 4 with no products	Inventory is Empty	Pass
TC18	Show sales summary	Complete sales of Rice, quantity 2	Most Sold Product and Quantity Sold displayed	Pass
TC19	Handle no sales data	Select option 5 before sales	No Sales Data Available	Pass
TC20	Exit application	Select menu option 6	Application Closed	Pass
TC21	Reject invalid menu choice	Enter choice 8	Invalid Choice. Enter 1 to 6	Pass
TC22	Handle non-numeric menu input	Enter letters instead of a number	Invalid Choice and menu is shown again	Pass